

HelixSkills

HelixSkills

CLI AI ajanları için anayasal temelli, yönetim odaklı beceri sistemi.

Özet

HelixSkills, CLI AI ajanları için geliştirilmiş bir beceri sistemi olup, Helix Constitution'u alt modül olarak devralır; böylece evrensel yönetim kuralları koşulsuz olarak geçerlidir. Yüklenebilir ajan becerileri, MCP araç sunucuları, Claude Code eklentileri ve yeniden kullanılabilir motorları kaydedilebilir, belgelenmiş bir katalog altında toplanır.

Kısa açıklama

HelixSkills, CLI AI ajanları için bir beceri sistemidir. Helix Constitution'u alt modül olarak entegre eder; böylece tüm evrensel kurallar geçerli olur ve ardından kaydedilebilir beceriler (eylem-ön ek sistemi, medya doğrulayıcı, çoklu kanal, oturum senkronizasyonu, işlenebilir öğe yaşam döngüsü ve daha fazlası), iki MCP araç sunucusu, iki Claude Code eklentisi ve yeniden kullanılabilir motorlar sunar.

Ayrıntılı açıklama

HelixSkills (skills deposu, Apache-2.0), CLI AI ajanları için bir beceri sistemi olup, alışılmışın aksine bilinçli bir sıralama değişikliğiyle başlar: Önce yönetim, sonra yetenek. constitution/ alt modülü olarak Helix Constitution'u devralır; böylece constitution/CLAUDE.md ve constitution/Constitution.md dosyalarındaki her evrensel kural, bir ajanın uymayı tercih edebileceği bir teamül olarak değil, proje ağacına fiziksel olarak

entegre edilmiş bir kural seti olarak koşulsuz geçerlidir. HelixSkills'u benimseyen bir ajan, anayasadan çıkamaz; kurallar kodla birlikte taşınır.

Çoğu “beceri çerçevesi” soyut kavramlarla iş yaparken, HelixSkills somut, kaydedilebilir ve yüklenebilir bir envanter sunar. Yedi anayasal beceri `register.sh` aracılığıyla yüklenir: `action-prefix-system`, `media-validator`, `multitrack`, `reporting-workable-items`, `scheduled-work-queue`, `session-sync` ve `workable-item-lifecycle` — orta ila ileri düzeyde bir yelpaze sunan bu beceriler, disiplinli eylem adlandırma, medya doğrulama ve bir iş biriminin tüm yaşam döngüsünü kapsar. Ek taslak beceriler (Android genel bakış, Java/Kotlin dili, Linux işletim sistemi) zaten indekslenmiş ve etkinleştirilmeyi bekliyor. İki MCP araç sunucusu (`media-validator`, `scheduled-work`), bu becerileri Model Context Protocol üzerinden ajanlara sunarken; iki Claude Code eklentisi (`helix`, `scheduled-work`) aynı yetenekleri doğrudan ajan çalışma zamanına entegre eder — tek bir beceri seti, ajanların hangi arayüzle iletişim kurduğuna bakılmaksızın erişilebilir.

Katalogun altında dört derinlik-1 yeniden kullanılabilir motor yer alır — `continuum` (uygulanmış), ayrıca `session_orchestrator`, `token_optimizer` ve `clickup_sync` (tasarım aşamasında) — becerilerin aynı altyapıyı yeniden icat etmesini önleyen ortak mekanizmalar. `token_optimizer` tek başına, vasic-digital ekosistem paketlerine (TOON, Embeddings, VectorDB, Normalize, conversation) ve HelixDevelopment'un LLMProvider'una uzanan açık bir bağımlılık grafiği bildirir; böylece çapraz depo bağlantıları örtük değil, denetlenebilir hale gelir. Tüm bunların çevresinde disiplinli bir belgeler sistemi bulunur: beceri kataloğu, otomatik oluşturulan beceri-grafiği indeksi, her depo için ayrıntılı sayfalar ve henüz tamamlanmamış noktaları açıkça belirten Gaps & Risks kaydı. Tüm sistem, dayanıklılık ve bölgesel erişim için GitHub, GitLab, GitFlic ve GitVerse üzerinde yansıtılmıştır.

İçerik

Neden inşa ettik

CLI AI ajanlarının tutarlı, yönetilebilir ve yeniden kullanılabilir yeteneklere ihtiyacı var — her biri kuralları yeniden icat eden geçici betiklere değil. HelixSkills, ajanlara paketlenmiş, kaydedilebilir ve ortak bir anayasaya bağlı bir beceri seti sunmak için inşa edildi; böylece davranış, bunu benimseyen her ajan ve projede tutarlı ve denetlenebilir kalıyor.

Neden oyunun kurallarını deęiřtiriyor

Ajan yeteneklerini taşıyabilir ve kurala uyumlu hale getiren řey, disiplin deęil, *yapısal tasarım*. Her beceri, bir anayasa alt modülü tarafından desteklenen, yönetilen, sürümlendirilmiş ve yüklenebilir bir birimdir — bir ajan bir beceriyi kaydettięi anda, aynı zamanda standart kural setini de miras alır ve sapma payı bırakmaz. Bu, daha önce pratikte mümkün olmayan bir řeyi mümkün kılıyor: Bir yeteneęi bir ajandan veya projeden dięerine taşımak ve onun aynı yönetim kurallarına baęlı olarak geldięini, her biri kuralları yeniden icat eden özelleřtirilmiş yapıřtırma betikleri yığını yerine standart arayüzler (MCP sunucuları ve Claude Code eklentileri) üzerinden ulařtığını bilmek.

Yenilikçi olan ne

- **Alt modül olarak Constitution:** Evrensel yönetim kuralları miras alınır, kopyalanmaz — ağaca monte edilir, böylece her kullanıcı ajan aynı standart kural setine baęlı kalır ve güncellemeler tek bir gerçek kaynaktan akar, onlarca eski kopyadan deęil.
- **Kendi kendini kaydeden birimler olarak sunulan beceriler** (`register.sh`) ve otomatik oluřturulan beceri-grafik dizinine entegre edilir; böylece katalog keřfedilebilir kalır ve yüklenenlerle asla senkronizasyonu bozulmaz.
- **Çoklu arayüz desteęi:** Aynı beceri seti, ajanlara MCP araç sunucuları ve Claude Code eklentileri üzerinden ulařır — bir kez yazın, ajanın kullandığı her çalışma zamanına hitap edin.
- **Ekosistem genelinde paylaşılan, derinlik-1 yeniden kullanılabilir motorlar** (`continuum`, `token_optimizer`, `session_orchestrator`, `clickup_sync`), her biri açık ve denetlenebilir çapraz repo baęımlılık bildirimleri taşıır; gizli baęlılıklar yerine.

En büyük teknik zorluklar ve çözümlerimiz

- **Birçok beceri ve ajan arasında ajan davranışını tutarlı ve kurala uyumlu tutmak** — yönetişimin her beceri için yeniden uygulanması zamanla sapmaya yol açar. Çözüm: Helix Constitution'u alt modül olarak monte etmek, böylece `constitution/CLAUDE.md` ve `constitution/Constitution.md` içindeki kurallar koşulsuz olarak uygulanır ve tek bir kaynaktan güncellenir; kopyalanıp unutulmaz.
- **Büyüyen bir beceri setini yüklenebilir ve keřfedilebilir kılmak** — kimsenin bulamadığı veya yükleyemedięi bir katalog işe yaramaz. Çözüm: Her beceri için `register.sh` kaydı ile yükleme sırasında becerilerin entegre edilmesi, ayrıca

otomatik oluşturulan INDEX beceri grafiği ve repo bazlı detaylı belgelerle keşfin gerçek durumu otomatik olarak takip etmesi.

- **Farklı çalışma zamanları kullanan ajanlara ulaşmak** — aynı yetenek her ana bilgisayar için yeniden inşa edilmemeli. Çözüm: Tek bir beceri setinin hem MCP araç sunucusu tanımları (constitution/mcp/ altında) hem de Claude Code eklentileri (constitution/plugins/ altında) aracılığıyla paketlenmesi; böylece tek bir uygulama farklı arayüzler üzerinden sunulur.

İçerik

Teknoloji Yığını

- **Shell (birincil dil)** — Aracıların bulunduğu her ortamda çalıştırma ve kayıt araçlarının ön yükleme gerektirmeden her yerde çalışması gerektiği için seçildi; register.sh ve install_upstreams dosyalarını güçlendirerek, başlangıç bağımlılıklarını sıfıra indirir ve taşınabilirliği sağlar.
- **Git alt modülleri** — Yönetişim kurallarının çoğaltılmadan devralınması için seçildi: Helix Constitution, constitution/ dizini altında canlı bir referans olarak bağlanır; böylece kural güncellemeleri tek bir işaretçi üzerinden yayılır, kopyala-yapıştır ve unutma sorunu ortadan kalkar.
- **Model Context Protocol (MCP)** — Aracılar için standart, çalışma zamanı bağımsız araç arayüzü olarak seçildi; constitution/mcp/ altında iki MCP sunucusu (media-validator, scheduled-work) tanımlanarak beceriler çağrılabilir araçlar olarak sunulur.
- **Claude Kod eklentileri** — Aracı çalışma ortamına sıfır yapıştırma ile becerilerin doğrudan entegre edilmesi için seçildi; iki eklenti (helix, scheduled-work) constitution/plugins/ altında yer alır ve farklı bir ana makine için MCP yüzeyini yansıtır.
- **Yeniden kullanılabilir motorlar (continuum, token_optimizer, session_orchestrator, clickup_sync)** — Becerilerden bağımsız olarak ortak mekanizmaların ayrıştırılması ve projeler arası yeniden kullanım için seçildi; örneğin token_optimizer, vasic-digital paketlerine (TOON, Embeddings, VectorDB, Normalize, conversation) ve HelixDevelopment'un LLMProvider'una bildirilen bağımlılıklar aracılığıyla bağlanır, kod çoğaltma yerine.
- **Çoklu ana makine Git yansıtma (GitHub, GitLab, GitFlic, GitVerse)** — Tek bir ana makine kesintisi veya bölgesel engelin erişimi kesmemesi için seçildi; aynı depolama alanı dört farklı platformda canlı tutularak dayanıklılık ve erişilebilirlik sağlanır.

Durum ve Dürüstlük Notları

- **Durum: beta.** Yedi anayasa becerisi, iki MCP sunucusu ve iki eklenti yayınlandı; taslak beceriler indekslendi ve etkinleştirilmeyi bekliyor; dört derinlik-1 motorundan üçü (session_orchestrator, token_optimizer, clickup_sync) ise hâlâ tasarım aşamasında.
- README dosyasında proje helix_skills olarak anılıyor; resmi GitHub yolu ise HelixDevelopment/skills. README'deki izlenen bulgular sayısı, projenin kendi bildirdiği bir rakamdır.

Öncelik seviyesi: Helix-birincil.